Unlimited Attempts Allowed

Semantic Segmentation with Deep Learning

This assignment focuses on training and evaluating deep learning models for semantic segmentation using the 2D Semantic Labeling Potsdam dataset. The task involves preprocessing the dataset, implementing 5-fold cross-validation, and training some models.

Throughout the assignment, you will evaluate model performance by plotting training and validation loss and accuracy, visualizing model architectures, and comparing different methods.

Throughout the assignment, clearly document any assumptions made in your implementation, including all hyper-parameters. Additionally, include a complete Python script covering dataset processing, model training, and evaluation in the report to ensure transparency.

Note: If you are using Google Colab, ensure your code runs correctly before connecting to a GPU. Otherwise, you may waste your free GPU time without fully utilizing it. Since the models are trained for only 20 epochs, each model is expected to take less than 15 minutes to train on Google Colab using a single T4 GPU.

Regarding Google Colab, you may follow the official Colab welcome guide at https://colab.research.google.com/

Links to an external site.

.
For TensorFlow, you can refer to the tutorial available at https://www.tensorflow.org/tutorials/images/cnn

Links to an external site.

.

General description

In the shared Google Drive link

Links to an external site.

and in the Canvas Storage

Download Canvas Storage

, you will find a portion of the 2D Semantic Labeling Potsdam dataset, which has a pixel resolution of 5 cm. Each image tile is , representing the height, width, and number of bands. The six bands consist of five input features, including Red, Green, Blue, Infrared (IR), and Elevation, along with one target band containing pixel-wise **labels**. This means that the 6th channel (index 5) in the data contains the labels.

The target band includes six possible classes:

1. Impervious surface
2. Building
3. Tree
4. Low vegetation
5. Car
6. Clutter/Background

For more details, refer to the dataset description at: https://www.isprs.org/resources/datasets/benchmarks/UrbanSemLab/2d-sem-label-potsdam.aspx

Links to an external site.

This assignment description assumes you are using the TensorFlow (TF) library for training. However, you are free to use PyTorch if preferred.

Step 1- Dataset preparation:

You need to read the dataset containing 15,048 GeoTIFF files and randomly sample at least 5,000 images for training. This selection ensures feasibility when using free platforms such as Google Colab, but you may use more images if your system allows it.

You should then prepare the dataset for 5-fold cross-validation, and if using TensorFlow, generate five TFRecord files, each representing a single data fold. If using PyTorch, ensure proper data management to support the subsequent steps for training deep models.

You can access a sample Jupyter Notebook [ipynb

Download ipynb

, pdf

Download pdf

] that demonstrates how to copy a shared ZIP file into your Google Colab, extract it, and list its contents. Furthermore, if you are unfamiliar with GeoTIFF file formats, you can use another notebook [ipynb

Download ipynb

, pdf

Download pdf

] to read a sample GeoTIFF file stored here

Download here

, visualize it, and print its latitude and longitude center.

Report:

- Select an image from the dataset, excluding the one provided above (0000000224-0000042784.tif), and visualize its bands using three different representations: an RGB image, an elevation band, and a target label band. Be sure to include a colorbar and class names!

- Describe the process of splitting the dataset into five folds for cross-validation.
- Python script

Note: In all steps outlined below, train only one model for each step by utilizing the designated training folds for simplicity. For instance, for each step use Folds 1, 2, and 3 as the training data, while using Fold 4 for validation, and reserving Fold 5 as the test set.

Step 2- Training a simple model:

In this step, you will train a simple model using only the RGB and IR bands as input. While reading the dataset for training, make sure to shuffle the dataset, and apply data augmentation, including rotation and flipping, to improve generalization. The model should be trained for 20 epochs using categorical cross-entropy as the loss function. During training, the model that achieves the best performance on the validation set should be saved as the final model.

Report:

- Visualize the model architecture.
- Plot the training and validation loss, as well as the accuracy over these epochs
- Evaluate the final model on the test set and report both the accuracy and loss values.
- Python script

Step 3- Training an Encoder-Decoder model:

Train a model using all five input bands, RGB, IR, and Elevation, as features. The model should be trained for 20 epochs with categorical cross-entropy as the loss function. Throughout the training process, the model that achieves the best performance on the validation set should be saved as the final version.

Report:

- Visualize the model architecture.
- Plot the training and validation loss, as well as the accuracy over these epochs
- Evaluate the final model on the test set and report both the accuracy and loss values.
- Use the final model to predict the semantic map of an input. Visualize the result including an RGB image, an elevation band, a target label band, and a prediction map. Be sure to include a colorbar and class names!
- Python script

Grading criteria:

Your submissions will be graded based on the following criteria:

- Justification and quality of problem solution
- Clarity in description of solution/implementation
- Quality in critical assessment/evaluation of solution
- Discussion of results

Self-check

- Have you answered all questions to the best of your ability?
- Is all the required information on the front page, is the file name correct etc.?
- Anything else you can easily check? (details, terminology, arguments, clearly stated answers etc.?)

Do not submit an incomplete module! We are available to help you, and you can receive a short extension if you contact us.